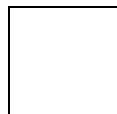


## PROJECT AND TEAM INFORMATION

### Project Title

*Data-Structure-Based USB Malware Activity Monitor Using C and C++ OOP*

### Student / Team Information

|                                                 |                                                                                                                                                                  |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Team Name:</i><br><i>DSCPP-III-2026-T370</i> |                                                                                                                                                                  |
| <b>Team member 1 (Team Lead)</b>                | <i>Divyanshu Panwar</i><br><i>2510370085@geu.ac.in</i><br><i>2028715</i><br> |
| <b>Team member 2</b>                            | <i>Abhay Rai</i><br><i>2510370531@geu.ac.in</i><br><i>20289610</i><br>      |
|                                                 |                                                                             |
|                                                 |                                                                             |

## PROPOSAL DESCRIPTION

### Motivation

USB drives remain one of the most common entry points for malware — autorun-based infections, BadUSB firmware attacks, and silent mass-copy data exfiltration all exploit the fact that most systems trust removable media by default. Manually watching for suspicious USB behavior does not scale, and commercial endpoint tools hide their internal logic behind proprietary engines. SentinelUSB aims to build a transparent, console-based monitor that detects, logs, and reacts to suspicious USB device and file activity in real time, while giving us hands-on practice applying core data structures (hash tables, linked lists, stacks, queues, trees) and C++ OOP (classes, inheritance, polymorphism) to a real security problem. The project focuses strictly on detection and monitoring — it does not create, execute, or distribute malicious code.

### State of the Art / Current solution

Commercial endpoint protection suites (Windows Defender, Symantec Endpoint Protection, McAfee) scan removable media using large signature databases and cloud-based heuristics. USBGuard (Linux) enforces device whitelisting/blacklisting policies at the kernel level using udev rules. Hardware USB firewalls and data-diode devices physically restrict device classes. These solutions are effective but heavyweight, often closed-source, and require background services or specialized hardware, which obscures how detection actually works. Our project simplifies this landscape by building a lightweight, console-based activity monitor using fundamental data structures — a hash-based device/signature registry, an event queue, and a category tree for scanned files — instead of a full antivirus engine, making the internal detection logic transparent for learning purposes.

### Project Goals and Milestones

#### General Goals:

- Design and implement a USB device and file-activity monitor using custom-built data structures in C/C++.
- Demonstrate how appropriate data structure choices (hash tables for signature/whitelist lookup, queues for event processing, stacks for quarantine rollback, trees for file categorization) improve detection speed and reliability.
- Apply C++ OOP principles to structure the system into reusable, maintainable classes (USBDevice, FileEvent, AlertManager, Logger).

#### Milestones:

1. Milestone 1 – Core Device Model: Define classes/structs for USBDevice, FileEvent, and Alert; implement basic device insertion/removal detection and event logging using arrays or linked lists.
2. Milestone 2 – Signature & Whitelist Indexing: Implement a hash table mapping device IDs (VID:PID) and known-malicious file hashes for O(1) average lookup against a whitelist/blacklist.
3. Milestone 3 – Event Processing Module: Implement a queue-based real-time event pipeline (FIFO) for incoming device/file events, plus a stack-based quarantine/undo feature for rolling back suspicious file moves.
4. Milestone 4 – Alerting & Reporting: Generate real-time alerts (mass file copy, unknown VID/PID, autorun.inf presence) and stock reports; test with simulated datasets and refine on edge cases.
5. Milestone 5 – Final Integration & Documentation: Combine modules into a single CLI (or optional simple dashboard), write documentation, and prepare a demo.

## Project Approach

We plan to build SentinelUSB using C for core data structure implementation (for low-level control and performance) combined with C++ for the OOP layer (encapsulating monitored entities as classes with inheritance, e.g., a base USBDevice class extended into MassStorageDevice, HIDDevice, etc.).

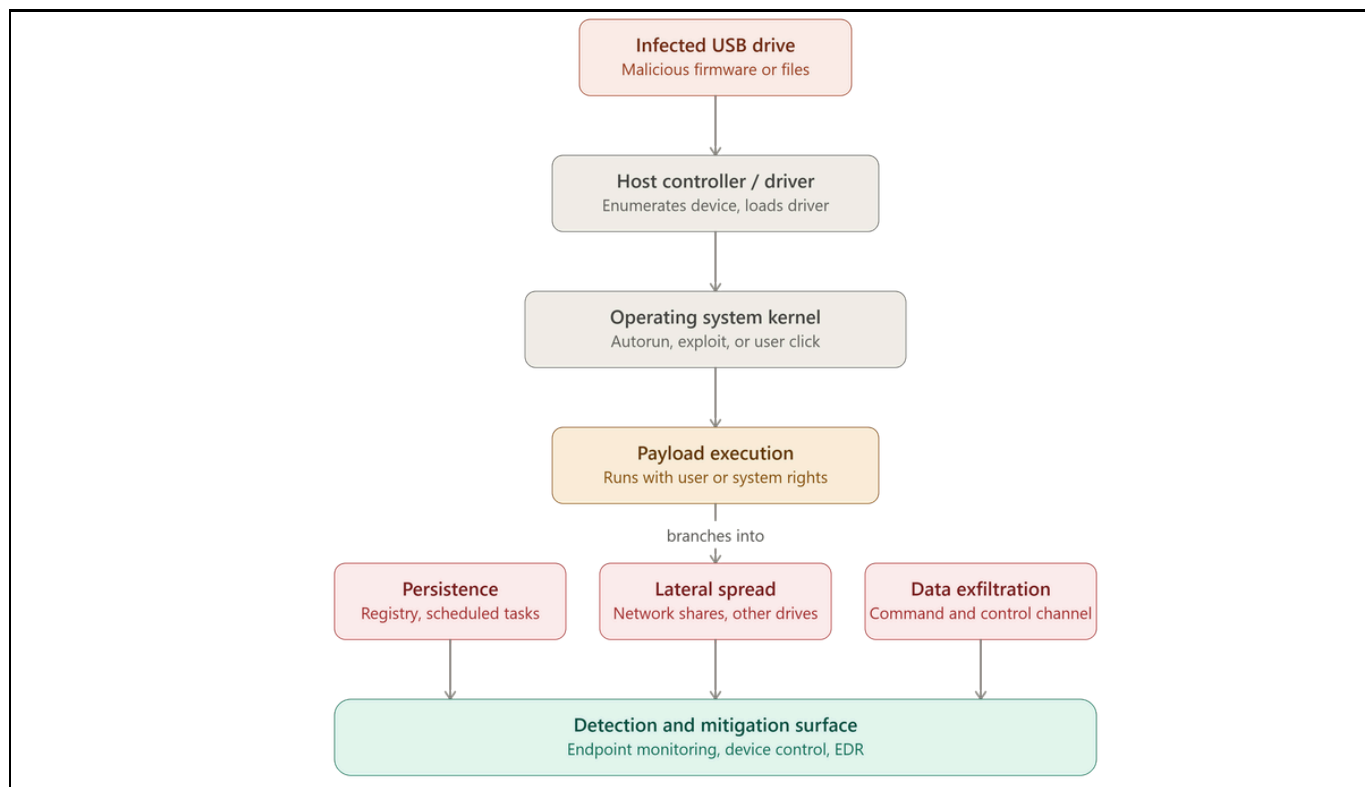
Key components:

- **Device Registry:** A hash table (custom implementation) mapping device VID:PID to device records and trust status for  $O(1)$  average lookup, insert, and delete.
- **Event Queue:** A FIFO queue to process incoming device-attach and file-system events in the order they occur.
- **Quarantine Stack:** A stack to support rollback/undo of the last  $N$  flagged file operations (e.g., isolating or restoring a suspicious file).
- **File-Category Tree:** A binary search tree (or trie) to organize scanned files by extension/type for fast range and prefix-based searches.
- **Sorting/Reporting:** Sorting algorithms (quicksort/mergesort) to generate reports like "most active devices" or "most frequent alert types."

Platform & Technologies:

- **Language:** C (data structures) + C++ (OOP interface layer)
- **Development environment:** VS Code / Code::Blocks with GCC/G++ compiler
- **Version control:** Git/GitHub for source management
- **Optional:** A simple command-line menu system; if time permits, a basic dashboard (Qt or Flask/C++ backend via REST) to visualize device and alert history
- **Testing:** Simulated event logs (CSV of device insert/remove and file-operation records) to validate detection logic and measure performance across data structure choices — no real malware samples are used or required

## System Architecture (High Level Diagram)



## Project Outcome / Deliverables

*The final deliverable is a fully functional, console-based SentinelUSB application implemented in C and C++ that detects USB device insertion/removal, flags suspicious file operations against a hash-based signature/whitelist, logs activity in real time, supports quarantine and rollback of flagged files, and generates activity reports (most active devices, alert-type breakdowns). The system will demonstrate practical use of hash tables, queues, stacks, and BSTs, along with a clean C++ OOP class hierarchy (USBDevice, FileEvent, AlertManager, Logger). Deliverables include the complete source code (hosted on GitHub), a sample simulated event-log CSV dataset used for testing, a project report documenting design decisions and data structure trade-offs, and a live demo showing core functionality (device monitoring, event processing, and reporting).*

## Assumptions

*We assume a single-machine, single-user console environment (no networked fleet monitoring), and that device/file events are supplied through simulated log or CSV input for development and testing rather than live low-level OS hooks, keeping the project portable and safe to demo. We assume dataset sizes are moderate (up to a few thousand logged events), so in-memory data structures are sufficient. The project is scoped strictly to detection, logging, and alerting on suspicious USB activity — it does not involve creating, executing, reverse-engineering, or distributing actual malicious code, and no real malware samples are used at any stage.*

## References

- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms* (for hash table, BST, and queue/stack theory)
- [cplusplus.com](http://cplusplus.com) / [cppreference.com](http://cppreference.com) — C++ STL and OOP documentation
- [GeeksforGeeks](http://GeeksforGeeks) — data structure implementation guides (hash tables, BSTs, queues)
- Deitel & Deitel, *C++ How to Program* (OOP design principles)
- USBGuard project documentation — policy-based USB device authorization (architecture reference)
- NIST SP 800-83 — *Guide to Malware Incident Prevention and Handling* (removable media security context)